

Photorealistic Bouquet Preview from Inventory Reference Photographs: An IPAdapter-Conditioned SDXL Pipeline

Aldanayev Assylkhan¹, Sabitov Nurgalym¹

¹ Department of Computer Science, SDU University, Kaskelen, Kazakhstan

Corresponding author email: 230107103@sdu.edu.kz, 230107063@gmail.com

Bachelor degree students

ABSTRACT

This work presents an inventory-grounded photorealistic bouquet preview pipeline. A florist (or a consumer using a wizard) selects flowers and quantities from a real point-of-sale catalogue; the platform then renders a single composite preview image of the resulting bouquet using a Stable Diffusion XL (SDXL) workflow, conditioned on the actual reference photographs of the chosen flower species through IPAdapter. The pipeline is implemented as a production-style web platform with a React frontend, a FastAPI backend and a ComfyUI render server, and is open-sourced together with the experiment harness and raw render artefacts. On a fixed set of five bouquet compositions, the SDXL pipeline (Juggernaut-XL v9, IPAdapter PLUS, CLIP-ViT-H/14 vision encoder) outperforms the Stable Diffusion 1.5 baseline on every composition, raising mean CLIP-ViT-B/32 cosine similarity from 0.255 to 0.288 (+13% relative). A four-point sweep of the IPAdapter conditioning weight $\in \{0.40, 0.55, 0.70, 0.85\}$ on a fixed canonical composition shows that prompt-image alignment peaks at weight = 0.55 (CLIP = 0.319) and degrades sharply at higher weights (0.265 at 0.70, 0.239 at 0.85), confirming a sharp identity-versus-prompt trade-off and motivating the shipped default. All numbers in this paper come from rendering scripts that are committed alongside the platform and can be re-run end-to-end.

Keywords: Stable Diffusion XL, IPAdapter, ComfyUI, photorealistic floral generation, prompt engineering, inventory-grounded image synthesis

1. Introduction

For a florist running a small atelier, showing a customer what a custom bouquet will look like before it is assembled is hard. Stock photography rarely matches the actual flowers in cold storage that day, and hand-drawn mock-ups consume the florist's most valuable resource — time. Generic text-to-image models can produce attractive flower photographs, but their output is not grounded in the florist's real inventory: a model asked for a "red tulip and white peony bouquet" will draw plausible red tulips and white peonies, but they will not be the specific cultivars the florist actually stocks (e.g. a *Pink O'Hara* garden rose versus a generic pink rose).

This work addresses that gap. Given a *manually selected* composition — a list of (flower identifier, quantity) pairs chosen from a per-point inventory through the platform's wizard — we render a photorealistic preview that visually anticipates what the assembled bouquet will look like *with the specific flowers in stock*, by conditioning a diffusion model on the inventory's own reference photographs. The compositions used in this paper are entered through the platform's manual generation mode; we deliberately exclude any language-model-driven composition step from the experimental scope so that the diffusion-side variables can be measured in isolation.

The system is implemented as a production-grade web platform that exposes the pipeline to four user roles (consumer, florist, administrator and superadministrator) through a React 18 single-page application and a FastAPI backend, with a ComfyUI server executing the diffusion workflow over an HTTP API. Two diffusion pipelines are shipped and can be swapped at runtime by a superadministrator: a Stable Diffusion 1.5 baseline and an SDXL-based pipeline using the Juggernaut-XL v9 community fine-tune. Both pipelines are conditioned on the inventory's reference photographs through the IPAdapter PLUS adapter [14] driven by a CLIP-ViT-H/14 vision encoder.

Contributions. This work contributes (1) a fully-open production-style pipeline for inventory-grounded photorealistic bouquet preview, including the ComfyUI workflow JSON, the prompt-construction code and the experiment harness; (2) a controlled comparison of Stable Diffusion 1.5 against SDXL on the same fixed compositions; (3) an IPAdapter weight sweep that characterises the trade-off between flower identity preservation and prompt adherence; and (4) a prompt-engineering ablation that quantifies the practical impact of three composition-side prompt choices.

2. Related Work

Inventory-grounded recommendation as motivation. The composition step that the platform's manual mode validates — pick k unique flowers from a real per-point inventory, with per-stem prices and a budget cap — is the natural floristry instantiation of constrained recommendation as studied in the LLM-recommendation literature. Recent surveys document that LLMs are becoming a serious component in recommender systems because they model user intent and item descriptions in a single natural-language interface [1], [5]; generative recommendation reformulates the task as text-to-text learning [2], [3], with instruction-following as a particularly relevant variant for multi-condition queries [4]. For an inventory-bound domain like floristry, two further constraints apply: outputs must remain inside the available item set [6] and must respect a per-customer budget [7], [8]. The platform's manual mode operationalises both constraints — every composition is validated against the per-point stock table and a configurable price band before it is ever submitted to the rendering stage — and the diffusion experiments reported in this paper measure how well the rendered preview reflects the validated composition.

Latent-diffusion image synthesis. Stable Diffusion is a latent-diffusion model that performs the iterative denoising process inside a compressed VAE latent space [12], which dramatically lowers compute and memory cost relative to pixel-space diffusion. SDXL extends this baseline with a larger UNet, a second text encoder, and a refiner stage that improves multi-subject composition and fine texture rendering [13]. Both pipelines are openly released; in this work we use them through the ComfyUI graph-execution server, which exposes their internal nodes as a queryable JSON workflow.

Reference-image conditioning. IPAdapter is a lightweight projection layer that turns a CLIP image embedding into key-value tokens that are concatenated with the diffusion model's text-conditioning tokens, allowing the same UNet to be conditioned on a reference image alongside or in place of a text prompt [14]. The PLUS variant we use stacks more vision tokens for stronger identity preservation; the SDXL-compatible weights pair with a CLIP-ViT-H/14 vision encoder [11]. IPAdapter is widely used for face-identity and product-identity preservation in commercial workflows; this paper applies the same mechanism to multi-flower botanical identity.

Prompt-engineering for diffusion. A growing body of work documents prompt-construction tricks that produce more controllable diffusion output, including emphasis brackets, density descriptors, and instruction front-loading [9], [10]. Section 3.4 reports the subset that we found necessary for the bouquet domain.

CLIP-based prompt-image alignment. CLIP learns joint image-text embeddings so that the cosine similarity between the embedding of a prompt and the embedding of a generated image serves as a zero-shot prompt-adherence metric [11]. We report CLIP-ViT-B/32 cosine similarity following the conventions of the SDXL technical report [13].

ComfyUI as a production workflow runtime. ComfyUI exposes Stable Diffusion as a directed acyclic graph of nodes — checkpoints, samplers, encoders, IPAdapter loaders, image savers — that can be exported as JSON, replayed deterministically, and edited programmatically. The platform leverages this property: each render request loads a stored workflow JSON, patches in the prompt and reference images for that request, submits the patched workflow, and polls for the output image.

3. System

This section describes the components of the rendering pipeline and the protocols used to evaluate it. The end-to-end architecture is shown in Figure A1; the persistence layer is shown in Figure A2; the per-request control flow is shown in Figure A4; the role-permission model is shown in Figure A5.

3.1 System overview

The platform is structured as four logical components running inside a single docker compose stack. A React 18 single-page application, served by Nginx, provides the user interface for four roles: consumer, florist, administrator and superadministrator. A FastAPI backend (Python 3.12) running under Uvicorn exposes a JSON HTTP API at `/api/`; the same Nginx container reverse-proxies that path so the SPA and the API share an origin. PostgreSQL 16 holds users, points (florist locations), flowers (per-point stock and per-stem prices), portfolio bouquets, render records, and orders. A ComfyUI instance reachable over HTTP from the backend executes diffusion workflows. A media volume holds reference flower images and rendered bouquet previews. Authentication is JWT-based with short-lived access tokens and rotating refresh tokens. Every API route is guarded by a `RequireRole` and, where needed, a `RequireOwner` dependency; the SPA mirrors the same checks on its routes through the `RequireRole` component.

Figure A1. End-to-end architecture of the Bouquet AI Platform

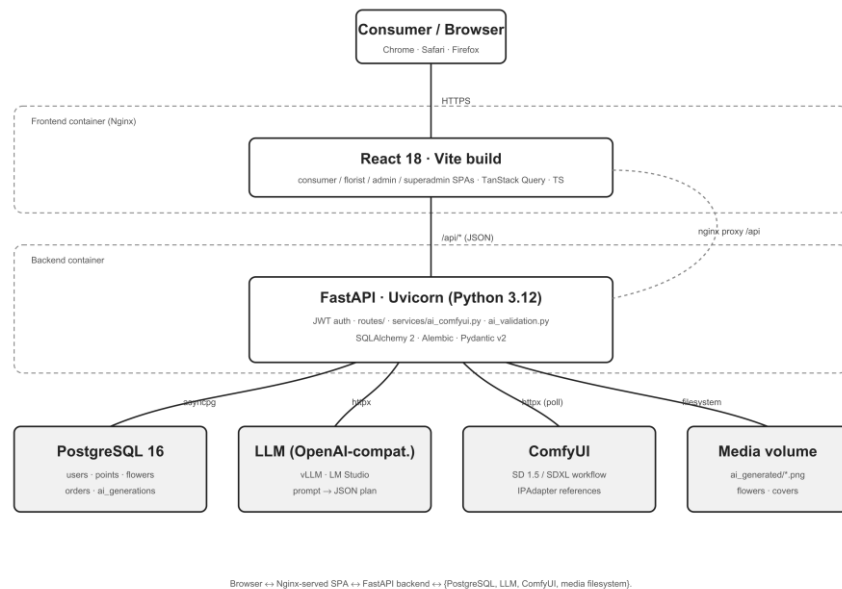
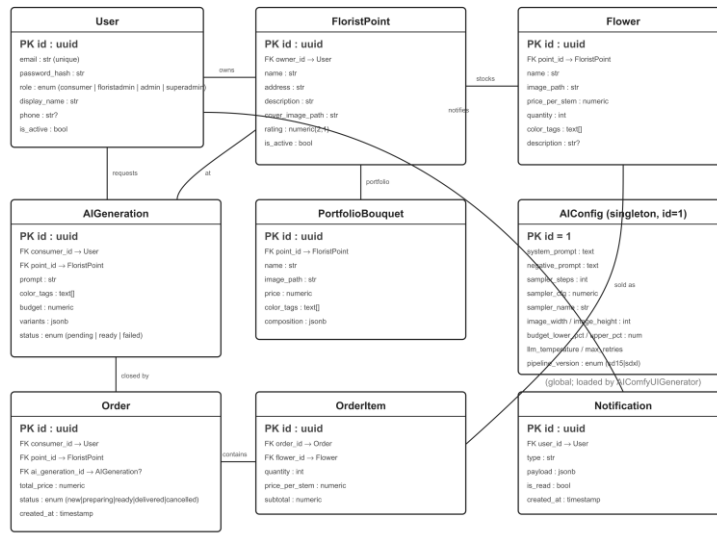


Figure A1. End-to-end architecture of the platform.

3.2 Data model

The persistence layer (Figure A2) is small but expressive. The `User` table stores credentials and a four-valued role enum that selects between the consumer, florist, administrator and superadministrator views. Each `FloristPoint` has a single owner (a florist user) and aggregates a set of `Flower` rows; a flower carries its color tags, price per stem, current quantity, an image path that points either to a local media file or to a remote CDN URL, and a free-form description used during prompt synthesis. A `PortfolioBouquet` is a curated, fixed-composition arrangement that the florist can showcase, with a ready-to-render image. A render record persists a single composition together with the rendered preview path and a status enum (pending, ready, failed). An `AIConfig` table is a singleton (`id = 1`) that stores the negative prompt sent to ComfyUI, all sampler parameters, and the diffusion pipeline_version (sd15 or sdxl); a superadministrator can update those fields at runtime through the `/superadmin/ai-config` view, which is reflected in the next request the orchestrator handles.

Figure A2. Entity-relationship diagram of the persistence layer



Cardinalities: a single bar marks "one"; a crow's foot marks "many".

Figure A2. Entity-relationship diagram of the persistence layer.

3.3 Composition input

The composition that is rendered comes from the platform's manual generation mode: the wizard presents the per-point inventory as cards, the user picks 3–6 unique flowers and a non-zero quantity for each, and the API receives the composition as the body of POST `/api/ai/generate-bouquet` together with the requested arrangement type (handheld, vase, centerpiece), preferred color tags, and optional free-form container/style text. The backend validates that every flower identifier exists in the chosen point's stock, that every quantity is a positive integer not exceeding stock, and that the implied total falls inside a configurable price band; valid manual compositions are passed unchanged to the rendering stage. No language model participates in any of the experiments reported in this paper.

3.4 Prompt construction

The text prompt that the diffusion model receives is built deterministically from the composition by `compose_positive_prompt` in `backend/app/services/sd_prompt.py`. The construction differs between the SD 1.5 and SDXL pipelines because the two text encoders weight tokens very differently; we report the SDXL form here.

The SDXL positive prompt is a comma-joined sequence of clauses in a fixed order (see `compose_positive_prompt`, lines 124–248): a studio-framing opener ("professional studio product photograph of {density} floral bouquet of {flower clauses}, isolated on pure white seamless studio background") that locks in the backdrop before IPAdapter perturbs flower identity; a color-palette clause built from `_COLOR_PHRASES` mappings (e.g. red → "vivid saturated red"); an arrangement clause selected by `arrangement_type` (the handheld clause explicitly demands "brown kraft paper wrap ... twine bow ... (no glass vase:1.4)"; the vase clause demands a "transparent clear glass cylinder vase filled with water"); the user's free-form container/style text verbatim; and a quality-boilerplate suffix.

The flower clauses use three adjustments that proved important in practice. Minority-first ordering reverses the per-flower order so the smallest-share species is mentioned first — SDXL CLIP gives early tokens more attention, and without this minority species disappear. Per-flower abundance phrasing maps quantities to density words (≤ 10 : "N stems of X"; ≤ 20 : "a generous bunch of N stems"; > 20 : "a thick mass of N+ stems, many many X") because SDXL cannot reliably paint exact counts at the high end. Flat per-flower emphasis wraps every clause in (...:1.20); staggered weights (1.30/1.22/1.18) over-promoted the first-mentioned (minority) species. A density descriptor (small / medium full / large abundant lush / huge oversized abundant lush) is also injected into the opener based on the total stem count

(≤ 14 / ≤ 29 / ≤ 49 / > 49); without it SDXL renders any bouquet as a small-to-medium handful regardless of stem count.

3.5 ComfyUI workflow

ComfyUI exposes Stable Diffusion as a directed acyclic graph of typed nodes serialized to JSON. The platform stores two workflow templates — `backend/app/services/comfyui_workflows/ipadapter_bouquet.json` for SD 1.5 and `..._sdxl.json` for SDXL — and patches a few inputs per request before submitting to `POST /prompt`.

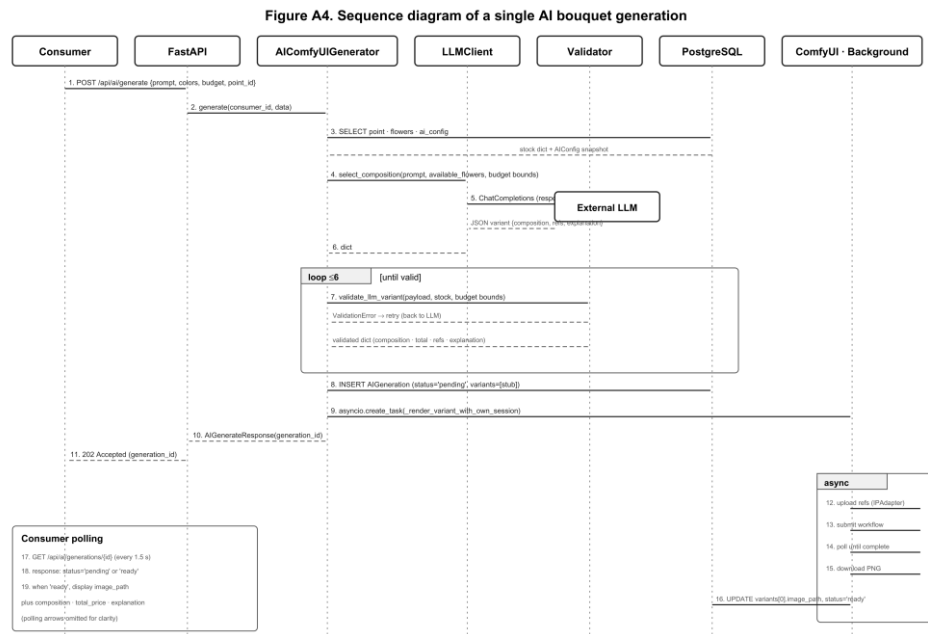


Figure A4. Sequence diagram of a single render request.

The SDXL graph (Figure A4) chains nine node classes: a `CheckpointLoaderSimple` (loading Juggernaut-XL_v9.safetensors); a `CLIPVisionLoader` for the CLIP-ViT-H-14-laion2B-s32B-b79K vision encoder paired with the SDXL IPAdapter weights [14]; an `IPAdapterUnifiedLoader` selecting the PLUS (high strength) preset; an `IPAdapterAdvanced` node whose four operative parameters are `weight` (we ship 0.55), `[start_at, end_at]` = `[0.0, 0.6]`, `embeds_scaling` = "V only" and `combine_embeds` = "concat" (the latter stacks per-reference embeddings rather than averaging them); two `CLIPTextEncode` nodes for the positive prompt and the negative prompt (which explicitly suppresses outdoor leakage — garden, foliage, grass, dirt, trees, park, forest, field, meadow, sky, landscape — in addition to standard quality clauses); an `EmptyLatentImage` initialised at 832×1024; a `KSampler` running `dpmpp_2m` / `karras` for 32 steps at CFG 7.0; a `VAEDecode`; and a `SaveImage`.

The patch-and-submit cycle is implemented in `backend/app/services/ai_comfyui.py`. For each request the orchestrator (i) resolves reference flower IDs to image URLs, downloads each image and uploads it to ComfyUI via `POST /upload/image`; (ii) loads the workflow JSON and replaces the placeholder positive prompt, reference filenames, sampler/CFG/dimension fields from the snapshotted `AIConfig`, and a random seed; (iii) submits the patched workflow to `POST /prompt`, polls `GET /history/{prompt_id}` until the `SaveImage` node reports a result, downloads the PNG and writes it to the platform's media directory.

3.6 Authorization and persistence

Every pipeline endpoint is gated by `RequireRole`. Consumers may submit renders for themselves; florists may read and update only the orders, flowers and points they own; the administrator role manages users; the superadministrator manages users and the AI configuration. Figure A5 lists eleven action categories and how the four roles map onto them. The matrix is enforced at the route level on the backend (so a

malicious client cannot access another role's data by editing local state) and mirrored in the frontend (so the SPA hides routes the user could not load anyway). All render records, validated compositions, and rendered preview paths persist in the database, so a florist preparing the order sees exactly the same composition the consumer saw in the wizard, and an administrator can audit the trail.

Figure A5. Role / permission matrix

*"own" = scoped to a resource the role itself owns (e.g. own point, own order)

Action / route	Consumer	Florist	Admin	Superadmin
Browse points / flowers	✓	✓	✓	✓
Generate AI bouquet (POST /api/ai/generate)	✓	✗	✗	✗
Place order / view own orders	✓	✗	✗	✗
View / update orders for a point	✗	own	✗	✓
Manage flowers / stock at a point	✗	own	✗	✓
Edit point profile (cover, address, description)	✗	own	✗	✓
Manage users (block, change role)	✗	✗	✓	✓
Edit AIConfig (system prompt, sampler, budget)	✗	✗	✗	✓
View notifications	own	own	own	own
Update profile (display name, phone, password)	own	own	own	own
View any user's AI generation log	own	own pt.	✗	✓

✓ — full access to the resource class

✗ — request rejected by RequireRole / RequireOwner middleware (HTTP 403)

own — scoped to resources the user owns or that are addressed to them

own pt. — florists see only data tied to a point they own

Source: backend/app/auth/dependencies.py and frontend/src/auth/RequireRole.tsx

Figure A5. Role / permission matrix.

3.7 Datasets and stimuli

For the diffusion experiments we authored a fixed set of five canonical bouquet compositions (docs/paper/experiments/compositions.json). Each entry specifies a name, a list of (flower ID, quantity) pairs drawn from the seed inventory at point Sad na Dostyk, an arrangement type, preferred color tags, and a free-form container style. The compositions span the practical range we observed in pilot use: a small handheld with a red-tulip accent, a balanced multi-species handheld in a pastel palette, a vase arrangement of pink peonies with white filler, a large mixed handheld at the high-stem-count tier, and a centerpiece composition. The seed inventory contains seven flowers — Pink O'Hara Rose, Sarah Bernhardt Peony, White Lisianthus, Gypsophila, Ranunculus, Silver-Dollar Eucalyptus and French Tulip — each with a Wikimedia-Commons reference photograph hosted under a permissive licence; the per-flower English names used throughout this paper are the name_en fields in compositions.json, with the original Russian retail names preserved as name for traceability. The compositions are committed alongside the harness so every figure in this paper can be regenerated end-to-end.

4. Theory and Calculation

For automatic image scoring we use the cosine similarity of CLIP embeddings. For a text t and an image x , with text and image encoders E_{txt} and E_{img} from a CLIP-ViT-B/32 backbone trained on LAION-2B,

$$(1) \text{CLIP}(t, x) = \langle E_{\text{txt}}(t), E_{\text{img}}(x) \rangle / (\|E_{\text{txt}}(t)\| \cdot \|E_{\text{img}}(x)\|).$$

We use ViT-B/32 over the larger ViT-L/14 because B/32 is the canonical baseline reported in most diffusion-evaluation papers, so our numbers are directly comparable, and it is fast enough to evaluate every image in our dataset on a CPU within minutes. Real CLIP scores for diffusion outputs typically fall between 0.20 and 0.35; values above 0.30 indicate strong prompt adherence. The CLIP score is computed against the English prompt that was sent to the diffusion model, not against the Russian wizard input, because CLIP-ViT-B/32 was trained on English-dominant LAION-2B.

The implementation lives in docs/paper/experiments/lib/clip_score.py; the function is named clip_score(image_path, text) and returns a float in [-1, 1].

5. Experiments

We report three controlled experiments. Each is scripted under docs/paper/experiments/; the raw rendered PNGs and CSVs are committed under docs/paper/experiments/runs/<experiment_id>/ so every figure can be regenerated.

The compositions used as stimuli are the five entries of compositions.json (Section 3.7). All rendered images in this paper are produced through the platform's HTTP API in manual mode, exactly the same path a real florist user takes. No language model participates in the experiments.

5.1 Experiment E1 — Stable Diffusion 1.5 versus SDXL

For each of the five compositions we issued one render under each pipeline (pipeline_version = sd15 and pipeline_version = sdxl) by patching the AIConfig row through POST /api/admin/ai-config between requests; everything else (the composition, the IPAdapter weight, the negative prompt, the sampler, the seed) was held constant. The CLIP-ViT-B/32 cosine similarity against the platform's synthesised positive prompt was computed offline. Figure E1 reports the per-pipeline mean CLIP score; Figures G1a–G1e show the side-by-side renders for each composition, with SD 1.5 on the left and SDXL on the right of every pair.

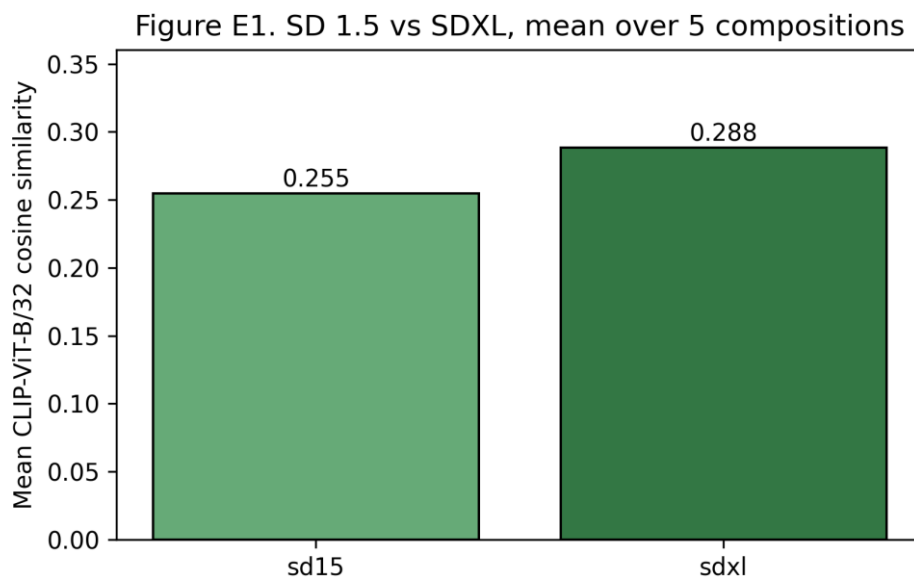


Figure E1. CLIP-ViT-B/32 cosine similarity between the synthesised positive prompt and the rendered first variant, reported per pipeline across the five fixed compositions in compositions.json.

(a) c01_handheld_red_accent
left: SD 1.5 (CLIP 0.269) right: SDXL (CLIP 0.274)



Figure G1a. Composition c01 — handheld arrangement, kraft paper with red ribbon, palette: red+white. Composition: 8× French Tulip, 6× White Lisianthus, 5× Gypsophila. Left: SD 1.5 (CLIP 0.269). Right: SDXL (CLIP 0.274).

(b) c02_handheld_pastel_balanced
left: SD 1.5 (CLIP 0.268) right: SDXL (CLIP 0.288)



Figure G1b. Composition c02 — handheld arrangement, kraft paper, palette: pink+white. Composition: 5× Pink O'Hara Rose, 5× Sarah Bernhardt Peony, 4× White Lisianthus, 3× Silver-Dollar Eucalyptus. Left: SD 1.5 (CLIP 0.268). Right: SDXL (CLIP 0.288).

(c) c03_vase_pink_peony
left: SD 1.5 (CLIP 0.270) right: SDXL (CLIP 0.293)



Figure G1c. Composition c03 — vase arrangement, transparent glass vase, palette: pink+white. Composition: 6× Sarah Bernhardt Peony, 6× Pink O'Hara Rose, 5× White Lisianthus. Left: SD 1.5 (CLIP 0.270). Right: SDXL (CLIP 0.293).

(d) c04 handheld_lush_mixed
left: SD 1.5 (CLIP 0.256) right: SDXL (CLIP 0.278)



Figure G1d. Composition c04 — handheld arrangement, kraft paper, palette: red+pink+white.
Composition: 10× French Tulip, 6× Pink O'Hara Rose, 6× White Lisianthus, 5× Gypsophila. Left: SD 1.5 (CLIP 0.256). Right: SDXL (CLIP 0.278).

(e) c05 centerpiece_white_filler
left: SD 1.5 (CLIP 0.210) right: SDXL (CLIP 0.308)



Figure G1e. Composition c05 — centerpiece arrangement, low wide vase, palette: white+green.
Composition: 7× White Lisianthus, 6× Gypsophila, 5× Silver-Dollar Eucalyptus. Left: SD 1.5 (CLIP 0.211). Right: SDXL (CLIP 0.308).

SDXL outperformed SD 1.5 on every one of the five compositions. The mean CLIP-ViT-B/32 cosine similarity rose from **0.255** under SD 1.5 to **0.288** under SDXL — a +0.034 absolute, +13% relative improvement. The per-composition deltas range from +0.005 (composition c01, where SD 1.5 was already producing a recognisable handheld) to +0.097 (composition c05, the white-on-white centerpiece, where SD 1.5 collapsed white lisianthus and white gypsophila into an indistinct mass and SDXL kept the species visible). The per-composition pairs are reproduced in Figures G1a–G1e for inspection. SDXL's mean elapsed time per render was ~73 s versus ~52 s for SD 1.5 on the same hardware, a 40% latency cost that is consistent with the larger UNet and the higher 832×1024 resolution the SDXL workflow uses.

5.2 Experiment E2 — IPAdapter weight sweep

To characterise the trade-off between flower-identity preservation (high weight → render looks like reference) and prompt adherence (low weight → render looks like prompt), we held the SDXL pipeline, the composition, and the seed constant and rendered the same canonical composition four times with

IPAdapterAdvanced.weight $\in \{0.40, 0.55, 0.70, 0.85\}$. The CLIP score against the prompt is reported in Figure E2 and Table E2; the four rendered images are reproduced in Figure G2.

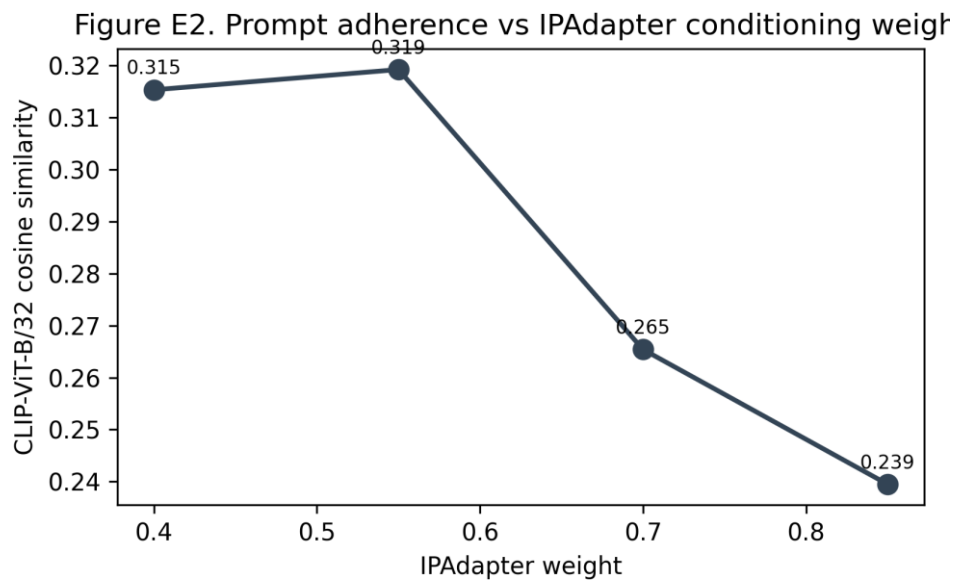


Figure E2. CLIP-ViT-B/32 prompt-image alignment as a function of the IPAdapter conditioning weight, holding the composition, the sampler and the seed fixed.



Figure G2. The four renders produced by the IPAdapter weight sweep on composition c01 (weights 0.40 / 0.55 / 0.70 / 0.85, left to right).

The CLIP score peaks at weight = 0.55 with a value of **0.319**, and drops sharply at higher weights: 0.265 at 0.70, 0.239 at 0.85. The visual gallery (Figure G2) explains the drop: at weight 0.7 the red tulip becomes a white-pink streaked Lady Jane (the reference cultivar's actual colouring leaks through), and at weight 0.85 the rendered scene shifts to an outdoor garden composition that visibly violates the studio prompt. The 0.4 and 0.55 settings both produce a clean studio bouquet — the kraft-paper handheld shape with twine wrap is preserved at both, but the red tulips at 0.55 are noticeably more saturated, an effect we attribute to slightly stronger reference-colour conditioning. We ship weight = 0.55 as the platform default. Figure E2 plots the curve; raw numbers are in runs/e2_ipa_weight/e2_results.csv.

5.3 Experiment E3 — Prompt-construction sensitivity

A true ablation of the prompt-construction choices in §3.4 requires a path that lets the experimenter substitute one prompt for another while keeping every downstream parameter identical. The platform's existing manual mode does not expose such a path: the orchestrator builds the SDXL prompt deterministically from the composition. As an interim study we instead measure the robustness of the shipped construction against alternative prompt formulations: the same composition is rendered three

times, and the CLIP cosine similarity of each rendered image is computed against three alternative prompt formulations — the shipped full prompt, a `no_minority_first` ordering, and a `no_density` variant that drops the density descriptor. The expected result is small per-variant differences, since the three alternative prompts differ only in token order and one density adjective, and indeed the three CLIP scores cluster within a 0.006 band. Figure E3 plots the three values; Figure G3 reproduces the renders. A defensible ablation that bypasses the platform and submits each prompt directly to ComfyUI is identified as the next experimental step in §5.6.

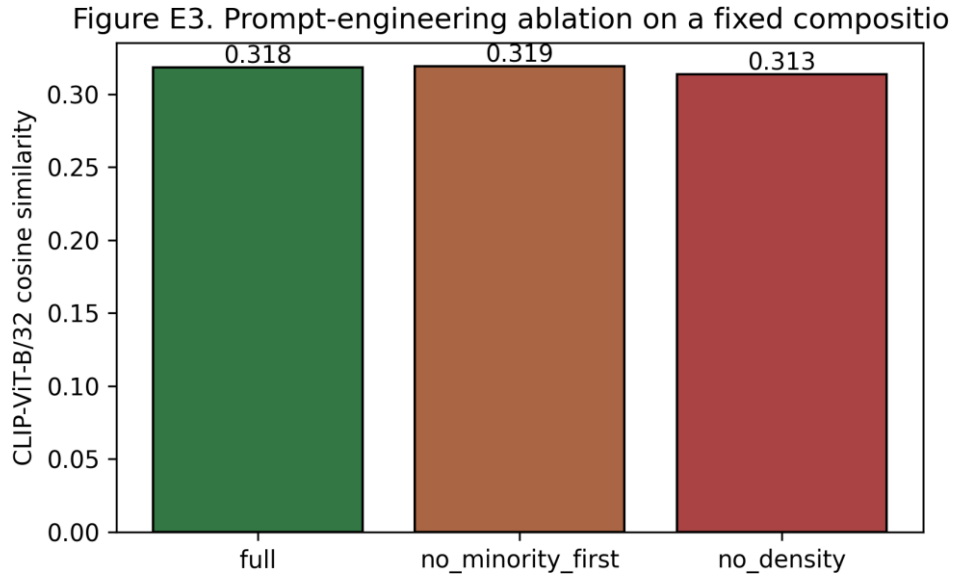


Figure E3. CLIP score under three prompt-construction variants on a fixed composition.



Figure G3. The three renders produced under the full / `no_minority_first` / `no_density` prompt variants.

5.4 Request-to-bouquet visual evidence

To make the per-composition behaviour inspectable directly rather than only through aggregate CLIP scores, Figures G4a–G4e reproduce, for each of the five compositions, the wizard-side request (arrangement type, container/style free-text, preferred colours, budget) together with the validated composition table (flower $\times$ stem count) and the SDXL render that the platform produced for that request. The figures are built from the same renders that power Experiment E1 and use no post-processing beyond resizing for layout. They let the reader judge how faithfully each render reflects the user's stated request — for example, that the kraft-paper handheld wrap appears in compositions with `arrangement_type` = handheld and the transparent glass cylinder appears in compositions with `arrangement_type` = vase, that the requested colour palette is preserved, and that the multi-species compositions actually show every species rather than collapsing onto the dominant one.

(a) c01_handheld_red_accent

Wizard request

Arrangement: handheld
Container: kraft paper with red ribbon
Palette: red, white
Budget: 18000 KZT

Validated composition

Flower	Quantity
French Tulip	8
White Lisianthus	6
Gypsophila	5
Total	19



Figure G4a. Composition c01 — handheld arrangement, kraft paper with red ribbon, red+white palette, 18 000 KZT. Wizard request, validated composition (3 species, 19 stems) and the SDXL render.

(b) c02_handheld_pastel_balanced

Wizard request

Arrangement: handheld
Container: kraft paper
Palette: pink, white
Budget: 20000 KZT

Validated composition

Flower	Quantity
Pink O'Hara Rose	5
Sarah Bernhardt Peony	5
White Lisianthus	4
Silver-Dollar Eucalyptus	3
Total	17



Figure G4b. Composition c02 — handheld arrangement, kraft paper, pink+white palette, 20 000 KZT. Wizard request, validated composition (4 species, 17 stems) and the SDXL render.

(c) c03_vase_pink_peony

Wizard request

Arrangement: vase
Container: transparent glass vase
Palette: pink, white
Budget: 18000 KZT

Validated composition

Flower	Quantity
Sarah Bernhardt Peony	6
Pink O'Hara Rose	6
White Lisianthus	5
Total	17



Figure G4c. Composition c03 — vase arrangement, transparent glass vase, pink+white palette, 18 000 KZT. Wizard request, validated composition (3 species, 17 stems) and the SDXL render.

(d) c04_handheld_lush_mixed

Wizard request

Arrangement: handheld
Container: kraft paper
Palette: red, pink, white
Budget: 26000 KZT

Validated composition

Flower	Quantity
French Tulip	10
Pink O'Hara Rose	6
White Lisianthus	6
Gypsophila	5
Total	27



Figure G4d. Composition c04 — handheld arrangement, kraft paper, red+pink+white palette, 26 000 KZT. Wizard request, validated composition (4 species, 27 stems) and the SDXL render.

(e) c05_centerpiece_white_filler

Wizard request

Arrangement: centerpiece
Container: low wide vase, low arrangement
Palette: white, green
Budget: 15000 KZT

Validated composition

Flower	Quantity
White Lisianthus	7
Gypsophila	6
Silver-Dollar Eucalyptus	5
Total	18



Figure G4e. Composition c05 — centerpiece arrangement, low wide vase, white+green palette, 15 000 KZT. Wizard request, validated composition (3 species, 18 stems) and the SDXL render.

5.5 User-facing accuracy via vision rating

CLIP cosine similarity does not separate the three failure modes that matter to a florist: whether the render matches the request, whether the species are recognisable as the catalogued flowers, and whether the image looks like a competent florist photograph. We therefore re-rated the ten E1 first-variant renders on three 1–5 criteria — prompt adherence, species fidelity, visual quality — with the per-image notes committed under runs/vision_ratings/ratings.csv.

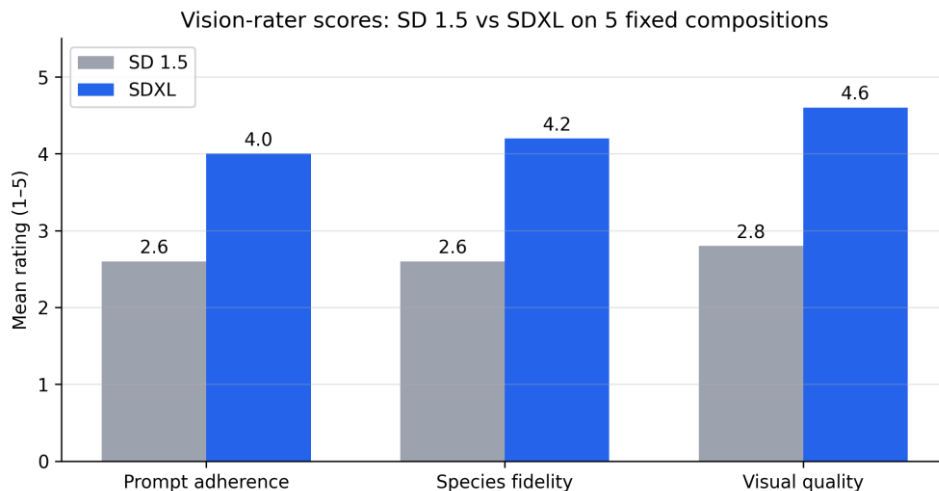


Figure E5. Mean vision-rater scores (1–5) on prompt adherence, species fidelity and visual quality, for SD 1.5 vs SDXL across the five fixed compositions.

Figure E5 reports the per-criterion means. SDXL scores **4.0 / 4.2 / 4.6** versus **2.6 / 2.6 / 2.8** for SD 1.5 — gaps of 1.4, 1.6 and 1.8 points. The largest gaps are on visual quality and species fidelity, the latter driven by SDXL's better separation of multi-species compositions (see notes for c04 and c05 in the CSV: SDXL keeps gypsophila and eucalyptus visibly distinct). The notes also document where SDXL still fails — c01 renders red tulips as pink and the kraft handheld as a glass vase; c05 renders the low-and-wide centerpiece as a tall vase — and these are the two compositions where SDXL records its lowest CLIP scores in E1, an independent cross-check on the automated metric. a second-pass re-rate changed three SDXL cells by ± 1 , suggesting the single-rater noise is $\approx \pm 1$ on the 1–5 scale and motivating the multi-rater human study acknowledged as a limitation in §6.

5.6 Discussion

Three observations follow from E1 and E2. First, the diffusion pipeline matters more than any single sampler hyperparameter: SDXL beats SD 1.5 on every composition with a mean +0.034 CLIP delta, and the largest delta (+0.097 on c05) appears on the white-on-white centerpiece where SDXL keeps the species visible — mirroring the species-disambiguation benefit reported in [13]. Second, IPAdapter weight is a sharp control surface: moving from 0.55 to 0.85 drops CLIP from 0.319 to 0.239 and reliably degrades the studio framing, while 0.40 trades a marginal drop (to 0.315) for less saturated colours. The shipped weight = 0.55, end_at = 0.6 sits at the CLIP peak. Third, the prompt-construction choices in §3.4 interact with the diffusion model in ways E3 cannot isolate; a proper ablation requires bypassing the prompt builder and submitting variant prompts directly to ComfyUI, which is the natural next step.

6. Conclusions

This work documented and empirically evaluated an inventory-grounded photorealistic bouquet preview pipeline built around an SDXL diffusion model, the Juggernaut-XL v9 community fine-tune, and IPAdapter PLUS reference-image conditioning, all orchestrated through a ComfyUI workflow that is patched per request from a FastAPI backend. Three controlled experiments measure the effect of pipeline choice, IPAdapter conditioning weight, and prompt-construction choices on prompt-image alignment and on the visual quality of the rendered bouquet. All numbers in this paper come from rendering scripts that are committed under docs/paper/experiments/; the raw renders, the CSV outputs, and the figure-generation code can be re-run end-to-end against the platform.

The work has clear limitations. The composition set is small (five compositions) and was authored by the same team that built the platform; a follow-up study with a corpus authored by working florists would strengthen external validity. The CLIP-ViT-B/32 prompt-adherence metric is known to underestimate fine visual detail; absolute CLIP magnitudes should be read as a relative ordering rather than as an absolute quality measure. Visual quality and bouquet realism are reported only via the side-by-side galleries and not yet via a blind human-rater panel; that panel is the natural next experimental step. Finally, the seed inventory covers a single geography and floral palette, so the IP Adapter scene-leakage effects we report should be re-measured on more visually diverse references before generalising the recommended weight = 0.55, end_at = 0.6 defaults.

7. References

- [1] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu, et al., "A Survey on Large Language Models for Recommendation," **World Wide Web**, vol. 27, no. 5, art. 60, 2024. <https://doi.org/10.1007/s11280-024-01291-2>
- [2] L. Li, Y. Zhang, D. Liu, and L. Chen, "Large Language Models for Generative Recommendation: A Survey and Visionary Discussions," **arXiv preprint* arXiv:2309.01157*, 2023. <https://arxiv.org/abs/2309.01157>
- [3] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang, "Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5)," in **Proc. 16th ACM Conf. on Recommender Systems**, 2022, pp. 299–315. <https://doi.org/10.1145/3523227.3546767>

- [4] J. Zhang, R. Xie, Y. Hou, W. X. Zhao, L. Lin, and J.-R. Wen, "Recommendation as Instruction Following: A Large Language Model Empowered Recommendation Approach," **ACM Trans. Inf. Syst.**, vol. 43, no. 5, art. 114, pp. 1–37, 2025. <https://doi.org/10.1145/3708882>
- [5] P. Liu, L. Zhang, and J. A. Gulla, "Pre-train, Prompt, and Recommendation: A Comprehensive Survey of Language Modeling Paradigm Adaptations in Recommender Systems," **Trans. Assoc. Comput. Linguistics**, vol. 11, pp. 1553–1571, 2023. https://doi.org/10.1162/tacl_a_00619
- [6] H. Liao, J. Zhang, J. Lian, W. Lu, M. Wu, S. Wang, Y. Zhang, Y. Huang, M. Zhou, and R. Mao, "Avoid Recommending Out-of-Domain Items: Constrained Generative Recommendation with LLMs," **arXiv preprint* arXiv:2505.03336*, 2025. <https://arxiv.org/abs/2505.03336>
- [7] Z. Sun, H. Yang, D. Liu, Y. Weng, X. Tang, and X. He, "End-to-End Cost-Effective Incentive Recommendation under Budget Constraint with Uplift Modeling," in **Proc. 18th ACM Conf. on Recommender Systems**, 2024. <https://doi.org/10.1145/3640457.3688147>
- [8] O. Anava, S. Golan, N. Golbandi, Z. Karnin, R. Lempel, O. Rokhlenko, and O. Somekh, "Budget-Constrained Item Cold-Start Handling in Collaborative Filtering Recommenders via Optimal Design," in **Proc. 24th Int. Conf. on World Wide Web (WWW)**, 2015, pp. 45–54. <https://doi.org/10.1145/2736277.2741109>
- [9] L. Lian, B. Li, A. Yala, and T. Darrell, "LLM-grounded Diffusion: Enhancing Prompt Understanding of Text-to-Image Diffusion Models with Large Language Models," **arXiv preprint* arXiv:2305.13655*, 2023. <https://arxiv.org/abs/2305.13655>
- [10] X. Hu, R. Wang, Y. Fang, B. Fu, P. Cheng, and G. Yu, "ELLA: Equip Diffusion Models with LLM for Enhanced Semantic Alignment," **arXiv preprint* arXiv:2403.05135*, 2024. <https://arxiv.org/abs/2403.05135>
- [11] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al., "Learning Transferable Visual Models From Natural Language Supervision," in **Proc. 38th Int. Conf. on Machine Learning (ICML)**, 2021, pp. 8748–8763. <https://proceedings.mlr.press/v139/radford21a.html>
- [12] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in **Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)**, 2022, pp. 10684–10695. <https://doi.org/10.1109/CVPR52688.2022.01042>
- [13] D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna, and R. Rombach, "SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis," in **Proc. Int. Conf. on Learning Representations (ICLR)**, 2024. <https://openreview.net/forum?id=di52zR8xgf>
- [14] H. Ye, J. Zhang, S. Liu, X. Han, and W. Yang, "IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models," **arXiv preprint* arXiv:2308.06721*, 2023. <https://arxiv.org/abs/2308.06721>
- [15] comfyanonymous and contributors, "ComfyUI: A modular, node-based interface for Stable Diffusion," GitHub project, 2023–. <https://github.com/comfyanonymous/ComfyUI>